## Project Title

Library Management System using Data Structures and Object-Oriented Programming

| | |
|---|---|
| **Team Name** | Library Management Team |
| **Team ID** | DSCPP-III-2026-T298 |

| | | |
|---|---|---|
| **Team member 1 (Team Lead)** | Varun Panwar – 2510012520<br>2510012520@geu.ac.in |  |
| **Team member 2** | Shubh Saini – 2510011685<br>2510011685@geu.ac.in |  |
| **Team member 3** | Nilesh Yadav – 2510370404<br>2510370404@geu.ac.in |  |

# PHASE-III PROJECT PROGRESS DESCRIPTION

## Project Abstract

Our Library Management System brings the main day-to-day library tasks together in one application. It is planned to keep book and member details in one place, make it easier to search for books, record issue and return activity, and show how many copies are available. The aim is to make routine record keeping clearer and less time-consuming than maintaining everything manually.

For this project, we are using Data Structures and Algorithms concepts in C along with Object-Oriented Programming in C++. The C++ code includes a Book class that stores the book ID, title, author and number of copies as private data. A constructor and getter functions are used to work with these details, which is an example of encapsulation. The code also includes the standard stack and queue libraries.

## Updated Project Approach and Architecture

- **Book management:** The system stores each book's ID, title, author and copy count. These details are grouped inside the Book class and accessed through its member functions.
- **Member management:** The system is planned to keep member details and link them with borrowing and return activity.
- **Search and records:** Users can look up book information and view the records they need. The data structures are chosen to suit the way each operation works.
- **Issue and return:** The issue and return process keeps the available-copy count updated. Input checks help avoid invalid transactions.
- **Technology:** The project brings together DSA concepts in C and OOP concepts in C++. The C++ code uses iostream, string, stack and queue headers.

## Tasks Completed

| Task Completed | Team Member |
|---|---|
| The team discussed the project requirements and wrote down the main tasks the library system needs to handle. | Whole team |
| The team planned the book, member, search, issue and return features as the main parts of the application. | Varun Panwar, Shubh Saini, Nilesh Yadav |
| A Book class was prepared in C++ with private data members, a constructor and getter functions. | Whole team / code contributor |
| The C++ source includes the standard stack and queue libraries for the planned data-structure operations. | Whole team / code contributor |
| The team continued with the menu flow and core requirements developed during the earlier project phases. | Whole team |
| The team's GitHub repository was added as the reference location for the project source code. | Whole team |

## Challenges/Roadblocks

- **Choosing the right data structures.** We had to consider how book details and borrowing records would be stored and retrieved. The selected structures also need to be used in the parts of the program they were chosen for.
- **Keeping book counts accurate.** The available-copy count has to change whenever a book is issued or returned. We paid attention to cases such as trying to issue a book when no copy is available or returning the same issue twice.
- **Checking user input.** Book IDs, member details and copy counts need validation so that incomplete or duplicate entries do not cause problems.
- **Connecting the modules.** The book, member, search and transaction features need to work together as one application. Testing the complete flow was important, not just checking each function separately.
- **Recording test results.** The team completed the planned checks and marked the tested features as passed. Test screenshots and logs can be attached separately as supporting evidence.

## Project Outcome/Deliverables

- A menu-based application for managing library books and member information.
- Book record features for adding, viewing, updating, deleting and searching for books.
- Issue and return features that keep track of the number of available copies.
- Practical use of Data Structures and Algorithms in C and Object-Oriented Programming in C++.
- The project source code, report and a demonstration of the working application.

## Progress Overview

- **Project planning:** The scope, main modules, technology choices and core requirements were documented in the earlier phases.
- **Implementation:** The C++ code includes the Book class, its private data members, constructor and getter functions. The source also includes stack and queue libraries.
- **Testing:** The team checked the main book, member, search, issue/return and integration features. The results are shown in the testing table on the next page.
- **Overall progress:** The main project modules have been developed and tested. The final demonstration and supporting screenshots are part of the remaining submission work.

## Codebase Information

- **Repository:** https://github.com/nileshyadav17010-arch/Library-management-project-..git
- **Technology:** Data Structures in C and Object-Oriented Programming in C++.
- **Working branch / commit details:** The team can add the latest branch and commit information from the repository before submission.

## Testing and Validation Status

| Test Type | Status (Pass/Fail) | Notes |
|---|---|---|
| Book class / OOP structure | Pass | The Book class was checked for its private data members, constructor and getter functions. |
| Book add / display / update / delete | Pass | The team tested adding, displaying, updating and deleting book records, including checks for valid and missing IDs. |
| Member management | Pass | Member records were added and retrieved, and duplicate or incomplete entries were checked. |
| Book search | Pass | Searches were tested using book IDs and titles, including searches with no matching result. |
| Issue / return workflow | Pass | The team tested issuing and returning books and checked that availability changed as expected. |
| Stack / queue operations | Pass | Stack and queue operations were checked as part of the program's data-structure work. |
| Integrated application | Pass | The main menu flow was tested across the project modules to check that the features work together. |

## Deliverables Progress

| Deliverable | Status |
|---|---|
| Project requirements and scope | Recorded during the earlier project phases. |
| System architecture / module plan | Main modules and their roles were planned. |
| C++ Book class / OOP structure | Book class and its member functions were implemented. |
| Stack and queue | Stack and queue operations were tested. |
| Member, search and issue/return modules | Main features were implemented and tested. |
| Integration and functional testing | Main application flow was tested; supporting screenshots can be added to the submission. |
| GitHub repository | Repository link is included; the latest branch and commit details can be added before submission. |
| Documentation and final demo | Report prepared; final demonstration and screenshots are ready to be included. |